

Arrays

Outline

- 1 Introduction
- 2 Arrays
- 3 Declaring Arrays
- 4 Examples Using Arrays
- 5 Passing Arrays to Functions
- 6 Sorting Arrays
- 7 Case Study: Computing Mean, Median and Mode Using Arrays
- 8 Searching Arrays
- 9 Multiple-Subscripted Arrays

Arrays Unit I Dr. Rabins Porwal

1

1 Introduction

- Arrays
 - Structures of related data items
 - Static entity – same size throughout program
 - Dynamic data structures

Arrays Unit I Dr. Rabins Porwal

2

2 Arrays

- Array
 - Group of consecutive memory locations
 - Same name and type
- To refer to an element, specify
 - Array name
 - Position number
- Format:


```
arrayname [ position number ]
```

 - First element at position 0
 - n element array named c:
 - c[0], c[1]...c[n - 1]

Name of array
(Note that all elements of this array have the same name, c)

↓

c[0]	-45
c[1]	6
c[2]	0
c[3]	72
c[4]	1543
c[5]	-89
c[6]	0
c[7]	62
c[8]	-3
c[9]	1
c[10]	6453
c[11]	78

↑

Position number of the element within array c

Arrays Unit I Dr. Rabins Porwal

3

2 Arrays

- Array elements are like normal variables


```
c[ 0 ] = 3;
printf( "%d", c[ 0 ] );
```

 - Perform operations in subscript. If x equals 3


```
c[ 5 - 2 ] == c[ 3 ] == c[ x ]
```

Arrays Unit I Dr. Rabins Porwal

4

3 Declaring Arrays

- When declaring arrays, specify
 - Name
 - Type of array
 - Number of elements


```
arrayType arrayName[ numberOfElements ];
```
 - Examples:


```
int c[ 10 ];
float myArray[ 3284 ];
```
- Declaring multiple arrays of same type
 - Format similar to regular variables
 - Example:


```
int b[ 100 ], x[ 27 ];
```

Arrays Unit I Dr. Rabins Porwal

5

4 Examples Using Arrays

- Initializers


```
int n[ 5 ] = { 1, 2, 3, 4, 5 };
int n[ 5 ] = { 0 }
```

 - If not enough initializers, rightmost elements become 0
 - All elements 0
 - If too many : a syntax error is produced (syntax error)
 - C arrays have no bounds checking
- If size omitted, initializers determine it


```
int n[ ] = { 1, 2, 3, 4, 5 };
// 5 initializers, therefore 5 element array
```

Arrays Unit I Dr. Rabins Porwal

6

```

1  /* fig06_08.c
2  Histogram printing program */
3  #include <stdio.h>
4  #define SIZE 10
5
6  int main()
7  {
8      int n[ SIZE ] = { 19, 3, 15, 7, 11, 9, 13, 5, 17, 1 };
9      int i, j;
10
11     printf( "%s%13s%17s\n", "Element", "Value", "Histogram" );
12
13     for ( i = 0; i <= SIZE - 1; i++ ) {
14         printf( "%7d%13d", i, n[ i ] );
15
16         for ( j = 1; j <= n[ i ]; j++ ) /* print one bar */
17             printf( "%c", '*' );
18
19         printf( "\n" );
20     }
21
22     return 0;
23 }

```

Outline

1. Initialize array
2. Loop
3. Print

Arrays Unit I Dr. Rabins Porwal

7

Element	Value	Histogram
0	19	*****
1	3	***
2	15	*****
3	7	*****
4	11	*****
5	9	*****
6	13	*****
7	5	*****
8	17	*****
9	1	*

Outline

Program Output

Arrays Unit I Dr. Rabins Porwal

8

4 Examples Using Arrays

- Character arrays
 - String **"first"** is really a static array of characters
 - Character arrays can be initialized using string literals

```
char string1[] = "first";
```

 - Null character `'\0'` terminates strings
 - `string1` actually has 6 elements
 - It is equivalent to

```
char string1[] = { 'f', 'i', 'r', 's', 't', '\0' };
```
 - Can access individual characters

```
string1[3] is character 's'
```
 - Array name is address of array, so `&` not needed for `scanf`

```
scanf( "%s", string2 );
```

 - Reads characters until whitespace encountered
 - Can write beyond end of array, be careful

Arrays Unit I Dr. Rabins Porwal

9

```

1  /* fig06_10.c
2  Treating character arrays as strings */
3  #include <stdio.h>
4
5  int main()
6  {
7      char string1[ 20 ], string2[] = "string literal";
8      int i;
9
10     printf( "Enter a string: " );
11     scanf( "%s", string1 );
12     printf( "string1 is: %s\nstring2 is: %s\n",
13           "string1 with spaces between characters is:\n",
14           string1, string2 );
15
16     for ( i = 0; string1[ i ] != '\0'; i++ )
17         printf( "%c ", string1[ i ] );
18
19     printf( "\n" );
20     return 0;
21 }

```

Outline

1. Initialize strings
2. Print strings
 - 2.1 Define loop
 - 2.2 Print characters individually
 - 2.3 Input string
3. Print string

Program Output

```

Enter a string: Hello there
string1 is: Hello
string2 is: string literal
string1 with spaces between characters is:
H e l l o

```

Arrays Unit I Dr. Rabins Porwal

10

5 Passing Arrays to Functions

- Passing arrays
 - To pass an array argument to a function, specify the name of the array without any brackets

```
int myArray[ 24 ];
myFunction( myArray, 24 );
```

 - Array size usually passed to function
 - Arrays passed call-by-reference
 - Name of array is address of first element
 - Function knows where the array is stored
 - Modifies original memory locations
- Passing array elements
 - Passed by call-by-value
 - Pass subscripted name (i.e., `myArray[ 3 ]`) to function

Arrays Unit I Dr. Rabins Porwal

11

5 Passing Arrays to Functions

- Function prototype

```
void modifyArray( int b[], int arraySize );
```

 - Parameter names optional in prototype
 - `int b[]` could be written `int []`
 - `int arraySize` could be simply `int`

Arrays Unit I Dr. Rabins Porwal

12

```

1  /* fig06_13.c
2  Passing arrays and individual array elements to functions */
3  #include <stdio.h>
4  #define SIZE 5
5
6  void modifyArray( int [], int ); /* appears strange */
7  void modifyElement( int );
8
9  int main()
10 {
11     int a[ SIZE ] = { 0, 1, 2, 3, 4 }, i;
12
13     printf( "Effects of passing entire array call "
14            "by reference:\n\nThe values of the "
15            "original array are:\n" );
16
17     for ( i = 0; i <= SIZE - 1; i++ )
18         printf( "%3d", a[ i ] );
19
20     printf( "\n" );
21     modifyArray( a, SIZE ); /* passed call by reference */
22     printf( "The values of the modified array are:\n" );
23
24     for ( i = 0; i <= SIZE - 1; i++ )
25         printf( "%3d", a[ i ] );
26
27     printf( "\n\nEffects of passing array element call "
28            "by value:\n\nThe value of a[3] is %d\n", a[ 3 ] );
29     modifyElement( a[ 3 ] );
30     printf( "The value of a[ 3 ] is %d\n", a[ 3 ] );
31     return 0;
32 }

```

Outline

1. Function definitions
2. Pass array to a function
 - 2.1 Pass array element to a function
3. Print

Entire arrays passed call-by-reference, and can be modified

Array elements passed call-by-value, and cannot be modified

Dr. Rabins Porwal

13

```

33
34 void modifyArray( int b[], int size )
35 {
36     int j;
37
38     for ( j = 0; j <= size - 1; j++ )
39         b[ j ] *= 2;
40 }
41
42 void modifyElement( int e )
43 {
44     printf( "Value in modifyElement is %d\n", e * 2 );
45 }

```

Outline

- 3.1 Function definitions

Effects of passing entire array call by reference:

The values of the original array are:

0 1 2 3 4

The values of the modified array are:

0 2 4 6 8

Effects of passing array element call by value:

The value of a[3] is 6

Value in modifyElement is 12

The value of a[3] is 6

Program Output

Dr. Rabins Porwal

14

Array Operations

- Traversal – the operation of visiting/ printing all elements of an array
- Insertion & Deletion – the operation of adding and removing an element from an array
- Sorting – the operation of rearranging the elements of an array in some specified order
- Searching – the operation of searching an element in an array

Outline

Arrays

Unit I

Dr. Rabins Porwal

Dr. Rabins Porwal

15

Algorithm for Array Traversal

Let *arr* be a linear array with lower bound LB and upper bound UB. The following algo traverses *arr* applying an operation PROCESS to each element of *arr*.

1. Initialize counter

Set counter = LB
2. Repeat steps 3 and 4 while counter <= UB
3. Visit element

Apply PROCESS to *arr*[counter]
4. Increase counter

Set counter = counter + 1
5. Exit

Outline

Arrays

Unit I

Dr. Rabins Porwal

Dr. Rabins Porwal

16

Insertion & Deletion

- Insertion – the operation of adding another element to the array
 - At the end of the array, provided there is enough space to accommodate additional elements in the array
 - In the beginning or middle of the array
 - All or half of the elements must be moved downwards to new locations to accommodate the new element and retain the order of other element.
- Deletion – the operation of removing an element from the array
 - Similarly, deleting the element from the end of an array is not a problem, but deleting from the middle requires movement of each element upwards in order to fill up the void in the array.

Outline

Arrays

Unit I

Dr. Rabins Porwal

Dr. Rabins Porwal

17

Algorithm for Insertion

INSERT(A, N, K, ITEM)

(where N is the number of items, K is the +ve integer s. t. K ≤ N. This algorithm inserts an element ITEM into Kth position of linear array A.)

1. Initialize counter

Set J = N
2. Repeat steps 3 and 4 while J ≥ K
3. Move Jth element downward

Set A[J+1] = A[J]
4. Decrease counter

Set J = J-1

End of step2 loop
5. Insert element

Set A[K] = ITEM
6. Reset N

Set N = N+1
7. Exit

Outline

Arrays

Unit I

Dr. Rabins Porwal

Dr. Rabins Porwal

18



19

Algorithm for Deletion

DELETE(A, N, K, ITEM)
(where N is the number of items, K is the +ve integer s. t. $K \leq N$. This algorithm deletes K^{th} element from linear array A.)

1. Set ITEM = A[K]
2. Repeat for J = K to N-1
(Move J+1 element upward)
Set A[J] = A[J+1]
End of loop
3. Reset the number N of elements in A
Set N = N-1
4. Exit

Arrays

Unit I

Dr. Rabins Porwal

19



20

6 Sorting Arrays

- Sorting data
 - Important computing application
 - Virtually every organization must sort some data
- Bubble sort (sinking sort)
 - Several passes through the array
 - Successive pairs of elements are compared
 - If increasing order (or identical), no change
 - If decreasing order, elements exchanged
 - Repeat
- Example:
 - original: 3 4 2 6 7
 - pass 1: 3 2 4 6 7
 - pass 2: 2 3 4 6 7
 - Small elements "bubble" to the top

Arrays

Unit I

Dr. Rabins Porwal

20



21

Algorithm for Bubble Sort

BUBBLE (A, N)
(This algorithm sorts the elements of an array A)

1. Repeat steps 2 and 3 for K=1 to N-1
2. Set PTR = 1 (Initialize pass pointer PTR)
3. Repeat while PTR <= N-K (Execute pass)
 - a. If A[PTR] > A[PTR+1], then
Interchange A[PTR] and A[PTR+1]
(End of if structure)
 - b. Set PTR = PTR+1
(End of Step3 inner loop)
(End of Step1 outer loop)
4. Exit

Arrays

Unit I

Dr. Rabins Porwal

21



22

7 Case Study: Computing Mean, Median and Mode Using Arrays

- Mean – average
- Median – number in middle of sorted list
 - 1, 2, 3, 4, 5
 - 3 is the median
- Mode – number that occurs most often
 - 1, 1, 1, 2, 3, 3, 4, 5
 - 1 is the mode

Arrays

Unit I

Dr. Rabins Porwal

22



23

Outline

```

1 /* Fig06_16.c
2  This program introduces the topic of survey data analysis.
3  It computes the mean, median, and mode of the data */
4 #include <stdio.h>
5 #define SIZE 99
6
7 void mean( const int [] );
8 void median( int [] );
9 void mode( int [], const int [] );
10 void bubbleSort( int [] );
11 void printArray( const int [] );
12
13 int main()
14 {
15     int frequency[ 10 ] = { 0 };
16     int response[ SIZE ] =
17     {
18         6, 7, 8, 9, 8, 7, 8, 9, 8, 9,
19         7, 8, 9, 5, 9, 8, 7, 8, 7, 8,
20         6, 7, 8, 9, 3, 9, 8, 7, 8, 7,
21         7, 8, 9, 8, 9, 8, 9, 7, 8, 9,
22         6, 7, 8, 7, 8, 7, 9, 8, 9, 2,
23         7, 8, 9, 8, 9, 8, 9, 7, 5, 3,
24         5, 6, 7, 2, 5, 3, 9, 4, 6, 4,
25         7, 8, 9, 6, 8, 7, 8, 9, 7, 8,
26         7, 4, 4, 2, 5, 3, 8, 7, 5, 6,
27         4, 5, 6, 1, 6, 5, 7, 8, 7 };
28
29     mean( response );
30     median( response );
31     mode( frequency, response );
32     return 0;
33 }

```

1. Function prototypes

1.1 Initialize array

2. Call functions mean, median, and mode

Dr. Rabins Porwal

23



24

Outline

```

34 void mean( const int answer[] )
35 {
36     int j, total = 0;
37
38     printf( "%s\n", "*****", " Mean", "*****" );
39
40     for ( j = 0; j <= SIZE - 1; j++ )
41         total += answer[ j ];
42
43     printf( "The mean is the average value of the data\n"
44            "items. The mean is equal to the total of\n"
45            "all the data items divided by the number\n"
46            "of data items ( %d ). The mean value for\n"
47            "this run is: %d / %d = %.4f\n",
48            SIZE, total, SIZE, ( double ) total / SIZE );
49 }
50
51 void median( int answer[] )
52 {
53     printf( "\n\n*****", " Median", "*****",
54            "The unsorted array of responses is" );
55
56     printArray( answer );
57     bubbleSort( answer );
58     printf( "\n\nThe sorted array is" );
59     printArray( answer );
60     printf( "\n\nThe median is element %d of\n"
61            "the sorted %d element array.\n"
62            "For this run the median is %d\n",
63            SIZE / 2, SIZE, answer[ SIZE / 2 ] );
64 }

```

3. Define function mean

3.1 Define function median

3.1.1 Sort Array

3.1.2 Print middle element

Dr. Rabins Porwal

24

```

25 }
26
27 void mode( int freq[], const int answer[] )
28 {
29     int rating, j, h, largest = 0, modeValue = 0;
30
31     printf( "\n%s\n%s\n%s\n",
32            "*****", " Mode", "*****" );
33
34     for ( rating = 1; rating <= 9; rating++ )
35         freq[ rating ] = 0;
36
37     for ( j = 0; j <= SIZE - 1; j++ )
38         ++freq[ answer[ j ] ];
39
40     printf( "%s11s19s\n%s4s\n%s4s\n",
41            "Response", "Frequency", "Histogram",
42            "1 1 2 2", "5 0 5 0 5 0 5 0" );
43
44     for ( rating = 1; rating <= 9; rating++ ) {
45         printf( "%8d\\11d", rating, freq[ rating ] );
46
47         if ( freq[ rating ] > largest ) {
48             largest = freq[ rating ];
49             modeValue = rating;
50         }
51
52         for ( h = 1; h <= freq[ rating ]; h++ )
53             printf( "*" );
54     }
55 }

```

Outline

3.2 Define function mode

3.2.1 Increase frequency[] depending on response[]

Notice how the subscript in **frequency[]** is the value of an element in **response[]** (**answer[]**)

Print stars depending on value of **frequency[]**

Dr. Robins Porwal

25

```

95 printf( "\n" );
96 }
97
98 printf( "The mode is the most frequent value.\n"
99        "For this run the mode is %d which occurred"
100        " %d times.\n", modeValue, largest );
101 }
102
103 void bubbleSort( int a[] )
104 {
105     int pass, j, hold;
106
107     for ( pass = 1; pass <= SIZE - 1; pass++ )
108
109         for ( j = 0; j <= SIZE - 2; j++ )
110
111             if ( a[ j ] > a[ j + 1 ] ) {
112                 hold = a[ j ];
113                 a[ j ] = a[ j + 1 ];
114                 a[ j + 1 ] = hold;
115             }
116 }
117
118 void printArray( const int a[] )
119 {
120     int j;
121
122     for ( j = 0; j <= SIZE - 1; j++ ) {
123
124         if ( j % 20 == 0 )
125             printf( "\n" );

```

3.3 Define bubbleSort

3.3 Define printArray

Bubble sort: if elements out of order, swap them.

Dr. Robins Porwal

26

```
26      printf( "%2d", a[ j ] );
27
28  }
29  }
30
31  *****
32  Mean
33  *****
34
35  The mean is the average value of the data
36  items. The mean is equal to the total of
37  all the data items divided by the number
38  of data items (99). The mean value for
39  this run is: 681 / 99 = 6.8788
40
41  *****
42  Median
43  *****
44
45  The unsorted array of responses is
46  7 8 9 8 7 8 9 8 9 7 8 9 5 9 8 7 8 7 8
47  6 7 8 9 3 9 8 7 8 7 7 8 9 8 9 8 9 7 8 9
48  6 7 8 7 8 7 9 8 9 2 7 8 9 8 9 8 9 7 5 3
49  5 6 7 2 5 3 9 4 6 4 7 8 9 6 8 7 8 9 7 8
50  7 4 2 2 5 3 8 7 5 6 4 5 6 1 6 5 7 8 7
51
52  The sorted array is
53  1 2 2 2 3 3 3 3 4 4 4 4 4 5 5 5 5 5 5 5
54  5 6 6 6 6 6 6 6 6 7 7 7 7 7 7 7 7 7 7
55  7 7 7 7 7 7 7 7 7 7 7 7 8 8 8 8 8 8 8
56  8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8
57  9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9
58
59  The median is element 49 of
60  the sorted 99 element array.
61  For this run the median is 7
62
63  Arrays
```

27

```

*****
Mode
*****
Response    Frequency    Histogram

                1    1    2    2
                5    0    5    0    5

1            1            *
2            3            ***
3            4            ****
4            5            *****
5            8            *********
6            9            **********
7            23           *************
8            27           *****************
9            19           *************

The mode is the most frequent value.
For this run the mode is 8 which occurred 27 times.

```

28



29

8 Searching Arrays: Linear Search and Binary Search

- Search an array for a *key value*
- Linear search
 - Simple
 - Compare each element of array with key value
 - Useful for small and unsorted arrays

29



30

Algorithm for Linear Search

LINEAR (A, N, ITEM, LOC)

(The algorithm finds the location LOC of ITEM in the array A, or sets LOC=0 if the search is unsuccessful)

1. [Insert ITEM at the end of A]

Set $A[N+1] = \text{ITEM}$

2. [Initialize counter]

Set $\text{LOC} = 1$

3. [Search for ITEM]

Repeat while $A[\text{LOC}] \neq \text{ITEM}$

Set $\text{LOC} = \text{LOC} + 1$

[End of loop]

4. [Successful ?]

If $\text{LOC} = N+1$, then Set $\text{LOC} = 0$

5. Exit

Arrays

Unit I

Dr. Rabins Porwal

30

8 Searching Arrays: Linear Search and Binary Search

- Binary search
 - For sorted arrays
 - Compares **middle** element with **key**
 - If equal, match found
 - If **key** < **middle**, looks in first half of array
 - If **key** > **middle**, looks in last half
 - Repeat
 - Very fast; at most n steps, where $2^n >$ number of elements
 - 30 elements array takes at most 5 steps
 - $2^5 > 30$ so at most 5 steps

Arrays Unit I Dr. Rabins Porwal

31

Binary Search Algorithm

BINARY (DATA, LB, UB, ITEM, LOC)

(The algorithm finds the location LOC of ITEM in the array DATA, or sets LOC=NULL if the search is unsuccessful)

1. [Initialize segment variables]
 - Set BEG = LB, END = UB and MID = INT ((BEG+END)/2)
2. Repeat steps 3 and 4 while BEG ≤ END and DATA[MID] ≠ ITEM
3. If ITEM < DATA[MID] then
 - set END = MID – 1
 - else
 - set BEG = MID + 1
4. Set MID = INT ((BEG+END)/2)
5. If DATA[MID] = ITEM, then
 - set LOC = MID
 - else
 - LOC = NULL
5. Exit

Arrays Unit I Dr. Rabins Porwal

32

Complexity of Binary Search

The complexity is measured by the number $f(n)$ of comparisons to locate ITEM in DATA where DATA contains n elements. It can be observed that each comparison reduces the sample size in half. Hence if $f(n)$ comparisons are required to locate ITEM, then

$$2^{f(n)} > n \text{ or equivalently } f(n) = \log_2 n + 1$$

That is, the running time for the worst case is approximately equal to $O(\log_2 n)$. It can also be shown that the running time for the average case is approximately equal to the running time for the worst case.

Arrays Unit I Dr. Rabins Porwal

33

Example on Complexity of Binary Search Algorithm

Suppose DATA contains 1000000 elements. It can be observed that $2^{10} = 1024 > 1000$ and hence $2^{20} > 1000^2 = 1000000$.

Hence, using the binary search algorithm, only about 20 comparisons will be required to find the location of an item in a DATA array with 1000000 elements.

Arrays Unit I Dr. Rabins Porwal

34

Limitations of Binary Search Algorithm

- Though this type of search algorithm is very efficient, but it has some limitations on arrays:
 - The list must be sorted.
 - One must have direct access to the middle element in any sublist
 - Keeping data in a sorted array is very expensive when there are many insertions and deletions.
- Alternative data structures:
 - Usage of linked list or Binary Search tree to store the data.

Arrays Unit I Dr. Rabins Porwal

35

9 Multiple-Subscripted Arrays

- Multiple subscripted arrays
 - Tables with rows and columns (**m** by **n** array)
 - Like matrices: specify row, then column

	Column 0	Column 1	Column 2	Column 3
Row 0	a[0][0]	a[0][1]	a[0][2]	a[0][3]
Row 1	a[1][0]	a[1][1]	a[1][2]	a[1][3]
Row 2	a[2][0]	a[2][1]	a[2][2]	a[2][3]

Array name: a
Row subscript: [0], [1], [2]
Column subscript: [0], [1], [2], [3]

Arrays Unit I Dr. Rabins Porwal

36

9 Multiple-Subscripted Arrays

- Initialization
 - Initializers grouped by row in braces
 - `int b[2][2] = { { 1, 2 }, { 3, 4 } };`
 - If not enough, unspecified elements set to zero
 - `int b[2][2] = { { 1 }, { 3, 4 } };`
- Referencing elements
 - Specify row, then column
 - `printf( "%d", b[0][1] );`

1	2
3	4

1	0
3	4

Arrays
Unit I
Dr. Rabins Porwal

37

Row Major and Column Major Order

- All elements of a matrix get stored in the memory in a linear fashion.
- Two ways to represent elements in computer's memory
 - Row major representation
 - Column major representation

Arrays
Unit I
Dr. Rabins Porwal

38

Address Calculation in single D array

- As the elements of the array A are stored in consecutive memory cells, the computer does not need to keep track of the address of every element of the array. It only needs to keep track the address of the first element of the array which is denoted by $base(A)$.
- Using this base address $base(A)$, the computer calculates the address of any element of the array by using the following formula:

$$addr(k^{th} \text{ element}) = base(A) + w \times (k-1)$$
 where w is the size of the data type of the array A.

Arrays
Unit I
Dr. Rabins Porwal

39

Address Calculation in 2-D array

- Using row major order, the formula to calculate address of $(j, k)^{th}$ element in a 2-D array A of size $m \times n$ is

$$addr(j, k) = base(A) + w \times [n(j-1) + (k-1)]$$
 where
 - $base(A)$ = the address of first element of the array, i.e., $A[0][0]$
 - w = word size
 - n = number of columns
- Using column major order, the formula to calculate address of $(j, k)^{th}$ element in a 2-D array A of size $m \times n$ is

$$addr(j, k) = base(A) + w \times [(j-1) + m(k-1)]$$
 where
 - $base(A)$ = the address of first element of the array, i.e., $A[0][0]$
 - w = word size
 - m = number of rows

Arrays
Unit I
Dr. Rabins Porwal

40

Example

Let
`int A[3][4] = { { 1, 2, 3, 4 }, { 5, 6, 7, 8 }, { 9, 10, 11, 12 } };`
 Assume base address of the array is 100.
 Since elements are integer type, $w = 2$
 Address of $(2, 3)^{th}$ element in the array A is

- Using row major order

$$addr(2, 3) = 100 + 2[4(2-1) + (3-1)]$$

$$112$$
- Using column major order

$$addr(2, 3) = 100 + 2[(2-1) + 3(3-1)]$$

$$114$$

Arrays
Unit I
Dr. Rabins Porwal

41

```

1 /* fig06_22.c
2  Double-subscripted array example */
3 #include <stdio.h>
4 #define STUDENTS 3
5 #define EXAMS 4
6
7 int minimum( const int[][EXAMS], int, int );
8 int maximum( const int[][EXAMS], int, int );
9 double average( const int[][EXAMS], int );
10 void printArray( const int[][EXAMS], int, int );
11
12 int main()
13 {
14     int student;
15     const int studentGrades[ STUDENTS ][ EXAMS ] =
16         { { 77, 68, 86, 73 },
17           { 96, 87, 89, 78 },
18           { 70, 90, 86, 81 } };
19
20     printf( "The array is:\n" );
21     printArray( studentGrades, STUDENTS, EXAMS );
22     printf( "\n\nLowest grade: %d\nHighest grade: %d\n",
23           minimum( studentGrades, STUDENTS, EXAMS ),
24           maximum( studentGrades, STUDENTS, EXAMS ) );
25
26     for ( student = 0; student <= STUDENTS - 1; student++ )
27         printf( "The average grade for student %d is %.2f\n",
28               student,
29               average( studentGrades[ student ], EXAMS ) );
30
31     return 0;
32 }

```

Each row is a particular student, each column is the grades on the exam.

`studentGrades[ ][ ]`

Outline

1. Initialize variables
- 1.1 Define functions to take double scripted
2. Call functions minimum, maximum, and average

Arrays
Unit I
Dr. Rabins Porwal

42

```

33
34 /* Find the minimum grade */
35 int minimum( const int grades[][ EXAMS ],
36             int pupils, int tests )
37 {
38     int i, j, lowGrade = 100;
39
40     for ( i = 0; i <= pupils - 1; i++ )
41         for ( j = 0; j <= tests - 1; j++ )
42             if ( grades[ i ][ j ] < lowGrade )
43                 lowGrade = grades[ i ][ j ];
44
45     return lowGrade;
46 }
47
48 /* Find the maximum grade */
49 int maximum( const int grades[][ EXAMS ],
50             int pupils, int tests )
51 {
52     int i, j, highGrade = 0;
53
54     for ( i = 0; i <= pupils - 1; i++ )
55         for ( j = 0; j <= tests - 1; j++ )
56             if ( grades[ i ][ j ] > highGrade )
57                 highGrade = grades[ i ][ j ];
58
59     return highGrade;
60 }
61
62 /* Determine the average grade for a particular exam */
63 double average( const int setOfGrades[], int tests )
64 {

```

Outline

3. Define functions

Dr. Rabins Porwal

43

```

65 int i, total = 0;
66
67 for ( i = 0; i <= tests - 1; i++ )
68     total += setOfGrades[ i ];
69
70 return ( double ) total / tests;
71 }
72
73 /* Print the array */
74 void printArray( const int grades[][ EXAMS ],
75                 int pupils, int tests )
76 {
77     int i, j;
78
79     printf( "          [0] [1] [2] [3]" );
80
81     for ( i = 0; i <= pupils - 1; i++ ) {
82         printf( "\nstudentGrades[%d] ", i );
83
84         for ( j = 0; j <= tests - 1; j++ )
85             printf( "%5d", grades[ i ][ j ] );
86     }
87 }

```

Outline

3. Define functions

Arrays

Unit I

Dr. Rabins Porwal

44

```

The array is:
          [0] [1] [2] [3]
studentGrades[0] 77  68  86  73
studentGrades[1] 96  87  89  78
studentGrades[2] 70  90  86  81

Lowest grade: 68
Highest grade: 96
The average grade for student 0 is 76.00
The average grade for student 1 is 87.50
The average grade for student 2 is 81.75

```

Outline

Program Output

Arrays

Unit I

Dr. Rabins Porwal

45



Applications & Limitations of Linear Arrays

- Linear arrays are used to represent all sorts of lists. They are used to implement other data structures such as stacks, queues, heaps etc.
- Limitations of Linear Arrays
 - The prior knowledge of number of elements in the linear array is necessary.
 - These are static structures, i.e., the memory used by them cannot be reduced or extended.
 - Since the elements are stored in consecutive locations, the insertions and deletions in these arrays are time consuming.

Arrays

Unit I

Dr. Rabins Porwal

46